第 10 节 实操任务1：定义 KBState 状态

> **学习目标**：`python -c "from workflows.state import KBState; print(KBState.__annotations__)"` 输出 7 个字段
> 目标文件：`workflows/state.py`

步骤 1：安装 LangGraph

```
pip install langgraph
```

验证安装成功：

```
python -c "import langgraph; print(f'langgraph {langgraph.__version__} 安
```

步骤 2：创建 workflows 目录

```
cd ~/ai-knowledge-base
mkdir -p workflows
touch workflows/__init__.py
```

步骤 3：用 AI 编程工具生成 state.py

以下代码可以用 **OpenCode**、**Claude Code**、**Cursor**、**Trae** 或**通义灵码**等任意 AI 编程工具生成。

提示词：

```
请帮我编写 workflows/state.py，定义 LangGraph 工作流的共享状态：

需求：
1. 使用 TypedDict 定义 KBState 类
2. 包含以下字段：
   - sources: list[dict] — 采集到的原始数据
   - analyses: list[dict] — LLM 分析后的结构化结果
   - articles: list[dict] — 格式化、去重后的知识条目
   - review_feedback: str — 审核反馈意见
   - review_passed: bool — 审核是否通过
   - iteration: int — 当前审核循环次数（最多 3 次）
   - cost_tracker: dict — Token 用量追踪
3. 每个字段加中文注释说明用途和数据格式
```

```
4. 遵循"报告式通信"原则：字段是结构化摘要，不是原始数据
```

生成的代码：（参考实现）

```
"""LangGraph 状态定义 — AI 知识库工作流的核心数据结构

所有节点共享同一个 KBState，通过 TypedDict 保证类型安全。
每个节点只修改自己负责的字段，实现职责隔离。
"""

from typing import TypedDict


class KBState(TypedDict):
    """知识库工作流的全局状态

    数据流向: sources → analyses → articles → review → save
    review_loop 是本项目的核心教学点——展示如何用条件边实现质量门控。

    Fields:
        sources: 原始采集数据，来自 GitHub API / RSS 等
        analyses: 经 LLM 分析后的结构化结果
        articles: 格式化、去重后的知识条目
        review_feedback: 审核 Agent 的反馈意见（中文）
        review_passed: 审核是否通过
        iteration: 当前审核循环次数（最多 3 次）
        cost_tracker: Token 用量追踪
    """

    sources: list[dict]         # 采集结果（报告式：摘要而非原始 HTML）
    analyses: list[dict]        # 分析结果（每条含 summary, tags, score）
    articles: list[dict]        # 知识条目（过滤 + 去重后的最终格式）
    review_feedback: str        # 审核反馈（具体改进建议，非空表示需修改）
    review_passed: bool         # 审核是否通过（条件边的判断依据）
    iteration: int              # 审核迭代次数（>= 3 时强制通过）
    cost_tracker: dict          # Token 统计 {prompt_tokens, completion_to
```

步骤 4：理解代码

如果你对这段代码有疑问，可以让 AI 编程工具解释：

> `请解释 workflows/state.py 的设计：`
> `1. 为什么用 TypedDict 而不是普通 dict？`
> `2. 为什么 sources 是 list[dict] 而不是 str？`
> `3. review_passed 为什么是 bool 不是 str？`
> `4. 为什么需要 iteration 字段？`

关键设计解读：

设计问题	答案
为什么用 TypedDict？	类型安全 + IDE 自动补全，LangGraph 要求 State 是 TypedDict
为什么 sources 是 `list[dict]`？	结构化数据方便下游解析，不需要再次理解格式
为什么 review_passed 是 `bool`？	条件边需要布尔判断，省去字符串解析
为什么需要 iteration？	防止审核循环无限执行，提供循环出口
为什么 cost_tracker 是 `dict`？	灵活存储多种计费字段，后续可扩展

步骤 5：验证 KBState

```
python -c "
from workflows.state import KBState

# 检查字段定义
annotations = KBState.__annotations__
print('KBState 字段：')
for name, type_hint in annotations.items():
    print(f'  {name}: {type_hint}')
print(f'\n共 {len(annotations)} 个字段')

# 创建一个实例
state: KBState = {
    'sources': [],
    'analyses': [],
    'articles': [],
    'review_feedback': '',
    'review_passed': False,
    'iteration': 0,
    'cost_tracker': {},
}
print(f'实例创建成功，iteration = {state[\"iteration\"]}')
"
```

期望输出：

```
KBState 字段：
  sources: list[dict]
  analyses: list[dict]
  articles: list[dict]
  review_feedback: <class 'str'>
  review_passed: <class 'bool'>
  iteration: <class 'int'>
```

```
  cost_tracker: <class 'dict'>

共 7 个字段
实例创建成功，iteration = 0
```

步骤 6：提交到 Git

```
git add workflows/__init__.py workflows/state.py
git commit -m "feat: add KBState definition for V3 LangGraph workflow"
```

完成！ 工作流的数据契约定义好了。KBState 的每个字段都是一份报告，这就是报告式通信的代码体现。下一步实现节点逻辑。